



# AUDIT REPORT

---

July 2026

For



# Table of Content

|                                                                                                                |    |
|----------------------------------------------------------------------------------------------------------------|----|
| Executive Summary                                                                                              | 03 |
| Number of Security Issues per Severity                                                                         | 05 |
| Summary of Issues                                                                                              | 06 |
| Checked Vulnerabilities                                                                                        | 07 |
| Techniques and Methods                                                                                         | 09 |
| Types of Severity                                                                                              | 11 |
| Types of Issues                                                                                                | 12 |
| Severity Matrix                                                                                                | 13 |
|  <b>Informational Issues</b> | 14 |
| 1. Compiler Version Should Be Fixed                                                                            | 14 |
| Functional Tests                                                                                               | 15 |
| Threat Model                                                                                                   | 16 |
| Automated Tests                                                                                                | 20 |
| Closing Summary & Disclaimer                                                                                   | 21 |



# Executive Summary

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Name</b>          | Konnex                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Protocol Type</b>         | Fixed-Supply ERC20 Token                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Project URL</b>           | <a href="https://konnex.world/">https://konnex.world/</a>                                                                                                                                                                                                                                                                                                                                                            |
| <b>Overview</b>              | <p>Konnex is a fixed-supply ERC20 token implemented using the OpenZeppelin ERC20 standard. The contract mints a total supply of 1,000,000,000 KNX to the deployer during deployment and does not provide any functionality for additional minting, burning, or administrative control thereafter. Its minimal design reduces the attack surface while maintaining full compliance with the ERC20 token standard.</p> |
| <b>Audit Scope</b>           | <p>The scope of this Audit was to analyze the Konnex Smart Contracts for quality, security, and correctness.</p>                                                                                                                                                                                                                                                                                                     |
| <b>Source Code link</b>      | <a href="https://github.com/vsmelov/konnex-contracts/tree/main/contracts">https://github.com/vsmelov/konnex-contracts/tree/main/contracts</a>                                                                                                                                                                                                                                                                        |
| <b>Branch</b>                | main                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Contracts in Scope</b>    | Konnex.sol                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Commit Hash</b>           | 012cc164f1c060c667631a832379c0ae42b4417e                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Language</b>              | Solidity                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Blockchain</b>            | Base                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Method</b>                | Manual Analysis, Functional Testing, Automated Testing                                                                                                                                                                                                                                                                                                                                                               |
| <b>Review 1</b>              | <a href="https://basescan.org/token/0x5a6ff6033d9b86aab5e00ccc91fd16234118e35a#code">https://basescan.org/token/0x5a6ff6033d9b86aab5e00ccc91fd16234118e35a#code</a>                                                                                                                                                                                                                                                  |
| <b>Review 1</b>              | 20th July 2026                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Updated Code Received</b> | NA                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Review 2</b>              | NA                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Fixed In</b>              | NA                                                                                                                                                                                                                                                                                                                                                                                                                   |



**Verify the Authenticity of Report on QuillAudits Leaderboard:**

<https://www.quillaudits.com/leaderboard>



# Number of Issues per Severity



|               |          |
|---------------|----------|
| Critical      | 0 (0%)   |
| High          | 0 (0%)   |
| Medium        | 0 (0%)   |
| Low           | 0 (0%)   |
| Informational | 1 (100%) |

|        |                    | Severity |      |        |     |               |
|--------|--------------------|----------|------|--------|-----|---------------|
|        |                    | Critical | High | Medium | Low | Informational |
| Issues | Open               | 0        | 0    | 0      | 0   | 0             |
|        | Acknowledged       | 0        | 0    | 0      | 0   | 1             |
|        | Partially Resolved | 0        | 0    | 0      | 0   | 0             |
|        | Resolved           | 0        | 0    | 0      | 0   | 0             |



# Summary of Issues

| Issue No. | Issue Title                      | Severity      | Status       |
|-----------|----------------------------------|---------------|--------------|
| 1         | Compiler Version Should Be Fixed | Informational | Acknowledged |



# Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations  
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**



# Techniques and Methods

**Throughout the audit of smart contracts, care was taken to ensure:**

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

**The following techniques, methods, and tools were used to review all the smart contracts:**

## Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

## Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



### Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

### Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

### Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



# Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

## ■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

## ■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

## ■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

## ■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

## ■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



# Types of Issues

## Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

## Resolved

These are the issues identified in the initial audit and have been successfully fixed.

## Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

## Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

# Severity Matrix

|            |        | Impact   |        |        |
|------------|--------|----------|--------|--------|
|            |        | High     | Medium | Low    |
| Likelihood | High   | Critical | High   | Medium |
|            | Medium | High     | Medium | Low    |
|            | Low    | Medium   | Low    | Low    |

## Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

## Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



# Informational Issues

## Compiler Version Should Be Fixed

**Acknowledged**

### Path

Konnex.sol

### Description

The contract uses a floating Solidity pragma:

```
>> pragma solidity ^0.8.28;
```

Using the caret (^) allows the contract to be compiled with any Solidity compiler version greater than or equal to 0.8.28 and lower than 0.9.0. Different compiler versions may introduce changes in code generation, optimizer behavior, bug fixes, or new compiler bugs, resulting in different deployment bytecode.

Although this is not a security vulnerability, using a floating pragma reduces build reproducibility and may cause discrepancies between the audited bytecode and the deployed bytecode if a different compiler version is used.

### Recommendation

```
pragma solidity 0.8.28;
```



# Functional Tests

Some of the tests performed are mentioned below:

- ✓ Verified the token name returns "Konnex", the symbol returns "KNX", and decimals returns 18
- ✓ Verified total supply equals  $1,000,000,000 * 10^{18}$  and is fully assigned to the deployer at deployment
- ✓ Verified no mint or burn function is externally reachable after deployment, so supply cannot change
- ✓ Verified tokens transfer between accounts with both balances updating by the exact amount
- ✓ Verified transfers exceeding the caller's balance or sent to the zero address revert
- ✓ Verified approve sets and overwrites the allowance, emits Approval, and rejects the zero address as spender
- ✓ Verified transferFrom moves tokens within the granted allowance and decrements it by the transferred amount
- ✓ Verified transferFrom beyond the allowance or the holder's balance reverts
- ✓ Verified an infinite allowance (`type(uint256).max`) is never decremented
- ✓ Verified totalSupply stays constant and the sum of all balances equals totalSupply across randomized transfer sequences



# Threat Model

## 1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

### 1.1 External Dependencies Table

| Dependency                                        | Type          | How It Is Used                                                     | Trust Assumption                                                                      | Associated Risks                                       |
|---------------------------------------------------|---------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------|
| ERC20 (OpenZeppelin v5)                           | Base contract | Entire token logic: balances, allowances, transfer, approve, _mint | OZ implementation is correct; no overrides in Konnex, so behavior is byte-for-byte OZ | Any OZ ERC20 bug propagates directly. None known in v5 |
| Context (OpenZeppelin)                            | Base contract | _msgSender() indirection                                           | No meta-tx forwarder wired in, so it returns raw msg.sender                           | None                                                   |
| IERC20 / IERC20Metadata / IERC20Errors (ERC-6093) | Interfaces    | Function signatures and custom error definitions                   | Match the ERC20 standard                                                              | None                                                   |
| Solidity ^0.8.28                                  | Compiler      | Checked arithmetic                                                 | No codegen bugs                                                                       | Floating pragma (informational)                        |





## 2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

### 2.1 Function Entry / Exit Table

Each function gets one table.

| Contract Name | Function                            | Category                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Konnex.sol    | constructor()                       | <p><b>What this function can do</b><br/>Set name "Konnex" and symbol "KNX"; mint 1,000,000,000 * 10<sup>18</sup> (1e27 wei) KNX to msg.sender</p> <p><b>What this function cannot/should not do</b><br/>Run again; mint after deployment; assign supply to any address other than the deployer</p> <p><b>Main invariant(s)</b><br/>Total supply fixed at 1e27 forever; _mint/_burn have no post-deployment call sites</p> <p><b>Access level</b><br/>Implicit restriction (once at deployment)</p> |
| Konnex.sol    | transfer(address to, uint256 value) | <p><b>What this function can do</b><br/>Move value from caller to to; emit Transfer</p> <p><b>What this function cannot/should not do</b><br/>Send to address(0) (ERC20InvalidReceiver); spend beyond caller balance (ERC20InsufficientBalance); modify totalSupply or allowances</p> <p><b>Main invariant(s)</b><br/>Balance conservation: sender debited and receiver credited by exactly value; no external calls or hooks, so no reentrancy surface</p> <p><b>Access level</b><br/>Public</p>  |



| Contract Name | Function                                              | Category                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Konnex.sol    | approve(address spender, uint256 value)               | <p><b>What this function can do</b><br/>Set allowance[caller][spender] to value (overwrite semantics); emit Approval</p> <p><b>What this function cannot/should not do</b><br/>Approve address(0) as spender; modify any balance</p> <p><b>Main invariant(s)</b><br/>Only the caller's own allowance entries mutate</p> <p><b>Access level</b><br/>Public</p>                                                                                                                         |
| Konnex.sol    | transferFrom(address from, address to, uint256 value) | <p><b>What this function can do</b><br/>Move value from from to to; decrement allowance[from][caller] by value</p> <p><b>What this function cannot/should not do</b><br/>Exceed allowance (ERC20InsufficientAllowance); use zero addresses; change supply</p> <p><b>Main invariant(s)</b><br/>Allowance strictly decreases by value, except type(uint256).max which is never decremented (v5 infinite-approval semantics)</p> <p><b>Access level</b><br/>Public (allowance-gated)</p> |



### 3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

#### 3.1 Asset-Holding Contracts

List only contracts that custody value.

| Contract   | Asset Type    | Custodied Assets                                                                                                                                                            |
|------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Konnex.sol | None intended | No receive/fallback, so plain ETH transfers revert. Any ERC20 (including KNX itself) transferred to the contract address is permanently stuck: there is no recover function |

#### 3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

| Contract   | Function     | Asset In                             | Asset Out                   | Caller   | Risk Notes                                                                                                 |
|------------|--------------|--------------------------------------|-----------------------------|----------|------------------------------------------------------------------------------------------------------------|
| Konnex.sol | construct or | None                                 | 1e27 KNX minted to deployer | Deployer | 100% supply concentration at genesis                                                                       |
| Konnex.sol | transfer     | KNX debit (caller)                   | KNX credit (to)             | Public   | Stuck-token footgun if to is the token contract or another non-recoverable address                         |
| Konnex.sol | transferFrom | KNX debit (from), allowance consumed | KNX credit (to)             | Public   | Infinite allowances persist indefinitely; approval phishing dominates real-world loss for tokens like this |



# Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



# Closing Summary

In this report, we have considered the security of Konnex. We performed our audit according to the procedure described above.

The assessment identified one Informational severity finding. No Critical, High, Medium, or Low severity vulnerabilities were identified. All identified issues have been documented and shared with the Konnex team.

# Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



# About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

**1M+**

Lines of Code Audited

**50+**

Chains Supported

**1500+**

Projects Secured

Follow Our Journey



# AUDIT REPORT

---

July 2026

For



Canada, India, Singapore, UAE, UK

[www.quillaudits.com](http://www.quillaudits.com)

[audits@quillaudits.com](mailto:audits@quillaudits.com)